A Practical Guide to Sampling from a Categorical Distribution in PyTorch

11 min read · Dec 5, 2024

whyamit404 Follow

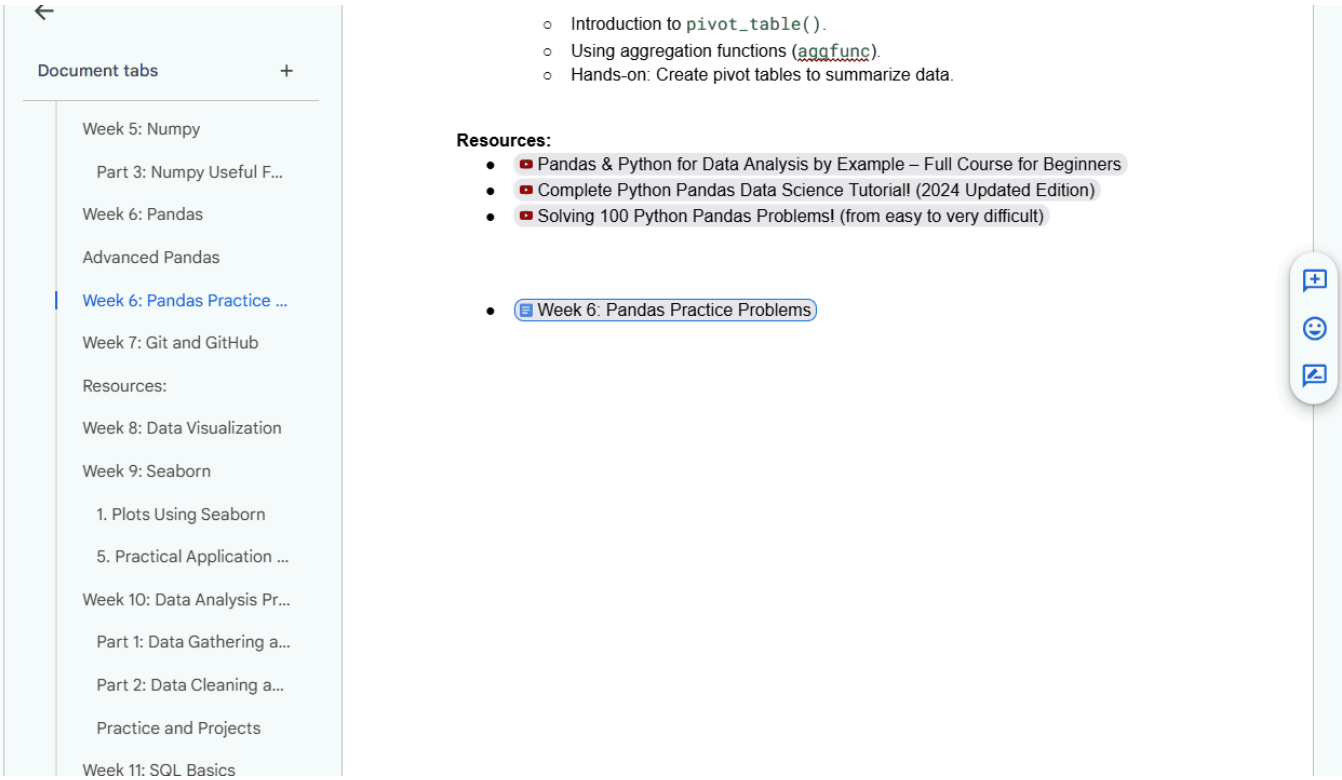
Listen Share

If you think you need to spend $2,000 on a 180-day program to become a data scientist, then listen to me for a minute.

I understand that learning data science can be really challenging, especially when you're just starting out, because you don't know what you need to know.

But it doesn't have to be this way.

That's why I spent weeks creating the perfect roadmap to help you land your first data science job.



Here's what it contains:

1. **A 42 weeks roadmap with study resources**
2. **30+ practice problems for each topic**
3. **A discord community**
4. **A resources hub that contains:**

- *Free-to-read books*
- *YouTube channels for data scientists*
- *Free courses*
- *Top GitHub repositories*
- *Free APIs*
- *List of data science communities to join*
- *Project ideas*
- *And much more…*

If this sounds exciting, you can grab it right now by clicking here.

Now let's get back to the blog:

. . .

> "Randomness isn't just chaos; it's a tool we wield with precision in machine learning."

Think about it: in our daily quest to teach machines how to make decisions, randomness often plays the role of a secret ingredient. One such form of randomness comes from **categorical distributions**.

Whether you're sampling an action in a reinforcement learning environment, selecting words for text generation, or even assigning classes during a stochastic process, categorical distributions are the workhorses of probabilistic decision-making.

But why PyTorch? If you've spent time in the machine learning trenches, you've probably noticed that PyTorch isn't just a library — it's like a Swiss Army knife for deep learning.

With its seamless handling of tensors, efficient GPU support, and a robust distributions module, PyTorch makes it ridiculously easy to work with categorical distributions.

Here's the deal: By the end of this guide, you'll know exactly how to sample from categorical distributions in PyTorch. And no fluff — I promise. You'll get actionable code snippets, examples rooted in practical applications, and tips to ensure you're doing this right the first time.

Let's get started.

. . .

Setting Up the Environment

Before we dive into the real action, let's make sure your system is ready to roll with PyTorch. You don't want to get halfway through an example only to find out something's broken — trust me, I've been there.

Install PyTorch

First things first, install PyTorch. If you already have it installed, verify the version — it's always better to work with the latest stable release.

Run these commands in your terminal:

```
pip install torch
python -c "import torch; print(torch.__version__)"
```

This does two things:

1. Installs PyTorch via pip.
2. Ensures your installation is working correctly by printing the version.

If you see something like 2.0.1 (or higher), you're good to go.

Import Necessary Libraries

Open in app ↗ Sign up Sign in

Medium Search

```
import torch
from torch.distributions import Categorical
```

Just two imports? Yes, but these two lines unlock a world of probabilistic modeling and sampling magic.

. . .

Understanding the Categorical Distribution in PyTorch

Let's cut to the chase: the torch.distributions.Categorical class is your one-stop solution for working with categorical distributions in PyTorch.

How Does It Work?

At its core, this class lets you define a categorical distribution using either:

1. **Probabilities (probs)**: A vector where each value represents the probability of an event.
2. **Logits (logits)**: A vector of unnormalized log-probabilities.

Why does PyTorch offer both? Simple: sometimes you work with raw probabilities, but other times (e.g., in neural networks) you only have logits. PyTorch gives you the flexibility to work with either, so you don't have to jump through hoops converting between them.

Creating a Categorical Distribution

Let me show you how to create one using probabilities:

```
# Define a probability vector
probs = torch.tensor([0.2, 0.5, 0.3])

# Create the categorical distribution
categorical_dist = Categorical(probs)

# Print the distribution
print("Categorical Distribution:", categorical_dist)
```

Here, probs represents a distribution over three events:

- Event 0 has a 20% chance.
- Event 1 has a 50% chance.
- Event 2 has a 30% chance.

You can use this distribution to sample events, calculate probabilities, and much more. Don't worry; we'll dive into all that in the next sections.

Using Logits Instead of Probabilities

Sometimes, probabilities aren't explicitly available, but logits are. Logits are simply the raw scores before they're normalized into probabilities using a softmax. Here's how you can work directly with logits:

```
# Define logits
logits = torch.tensor([0.1, 2.0, 0.5])

# Create a categorical distribution using logits
categorical_dist_logits = Categorical(logits=logits)

# Print the distribution
print("Categorical Distribution (Logits):", categorical_dist_logits)
```

What's happening here? PyTorch internally converts these logits into probabilities (via a softmax), but you don't have to worry about that — you can work directly with logits.

. . .

Sampling Basics

> "Life is all about making choices, and so are machines. Sampling is how they decide."

Let's get straight into it. When you define a categorical distribution using PyTorch, sampling is as simple as calling .sample(). Here's a quick example to see it in action:

```
import torch
from torch.distributions import Categorical

# Define a probability distribution
probs = torch.tensor([0.2, 0.5, 0.3])
categorical_dist = Categorical(probs)

# Take a sample
sample = categorical_dist.sample()

# Print the sampled index
print(f"Sampled index: {sample.item()}")
```

Here's what's happening:
The distribution (probs) has three events with probabilities 0.2, 0.5, and 0.3. When you call .sample(), PyTorch randomly picks one of these events based on the defined probabilities. The result is an integer index representing the selected event.

Sampling with Varying Probabilities

Now, let's see how the behavior changes when we tweak the probabilities. This is where it gets interesting.

```
# Define a new probability distribution
probs = torch.tensor([0.1, 0.1, 0.8])
dist = Categorical(probs)

# Sample multiple times
samples = [dist.sample().item() for _ in range(1000)]

# Count occurrences of each index
counts = torch.tensor(samples).bincount()
print("Sampled counts:", counts)
```

You'll notice that **index 2** (with a probability of 0.8) dominates the samples, while indices 0 and 1 appear much less frequently. This illustrates the power of categorical sampling: it respects the underlying probabilities, making it ideal for stochastic decision-making.

. . .

Using Logits Instead of Probabilities

You might be wondering, *"Why use logits when we have probabilities?"* Logits offer a numerical stability advantage, especially when dealing with very small or large probabilities. Instead of explicitly defining probabilities, logits let PyTorch calculate them internally using a softmax.

Here's how you can work with logits:

```
# Define logits (unnormalized log-probabilities)
logits = torch.tensor([0.1, 2.0, 0.5])
```

```
# Create the categorical distribution
dist = Categorical(logits=logits)

# Take a sample
sample = dist.sample()

# Print the sampled index
print(f"Sampled index: {sample.item()}")
```

In this example:

- Logits are unscaled values (e.g., raw outputs from a neural network).
- PyTorch automatically converts them into probabilities before sampling, so you don't have to worry about scaling or normalization.

This approach is especially useful in deep learning pipelines where logits naturally occur.

. . .

Batch Sampling

You've seen how to sample from a single distribution, but what if you need to sample for multiple distributions at once? PyTorch makes batch sampling incredibly efficient.

```
# Define batch probabilities
batch_probs = torch.tensor([
    [0.1, 0.7, 0.2],  # Distribution 1
    [0.3, 0.4, 0.3]   # Distribution 2
])

# Create the batch distribution
dist = Categorical(probs=batch_probs)

# Sample from both distributions
samples = dist.sample()

# Print the batch samples
print(f"Batch samples: {samples}")
```

Output:

The samples tensor contains one sampled index for each row (distribution) in batch_probs. Batch sampling is great for scenarios where you're working with multiple independent datasets or need to parallelize computations.

. . .

Gradient-Based Use Case

Now, here's where things get truly PyTorch-y. The Categorical distribution integrates seamlessly with PyTorch's autograd, allowing you to compute gradients and optimize models involving stochastic processes.

Get whyamit404's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

Let's walk through an example:

```
# Define probabilities with gradient tracking enabled
probs = torch.tensor([0.2, 0.5, 0.3], requires_grad=True)

# Create the distribution
dist = Categorical(probs)

# Compute the negative log-probability of a specific event
loss = -dist.log_prob(torch.tensor(1))  # Log probability of class 1

# Perform backpropagation
loss.backward()

# Print the gradient of the probabilities
print(f"Gradient: {probs.grad}")
```

What's happening here?

1. probs is a tensor with requires_grad=True, meaning PyTorch tracks its operations for gradient computation.
2. dist.log_prob(torch.tensor(1)) calculates the log-probability of sampling index 1.
3. loss.backward() computes the gradient of loss with respect to probs.

This is particularly useful in reinforcement learning, where you might optimize policies based on the gradients of sampled actions.

. . .

Practical Use Cases

"All theory is gray, my friend, but forever green is the tree of life." — Goethe.
And in machine learning, the tree of life is **practical application**. Let's shift from concepts to real-world scenarios where PyTorch's Categorical distribution shines.

a. Sampling for Reinforcement Learning

In reinforcement learning (RL), agents often need to decide actions probabilistically based on a policy. Categorical distributions are a natural fit for discrete action spaces. Let's break it down:

REINFORCE Example:

Here's how you'd sample an action and calculate its log-probability, a critical step in policy optimization:

```
import torch
from torch.distributions import Categorical

# Define policy probabilities for actions
probs = torch.tensor([0.2, 0.5, 0.3])

# Create the categorical distribution
dist = Categorical(probs)

# Sample an action
action = dist.sample()
```

```
# Calculate the log-probability of the sampled action
log_prob = dist.log_prob(action)

# Print results
print(f"Action: {action.item()}, Log Prob: {log_prob.item()}")
```

What's Happening Here?

1. The probs tensor defines a policy over three discrete actions.
2. The dist.sample() method selects an action based on the probabilities.
3. The log_prob() method calculates the log-probability of the sampled action, which is essential for updating the policy in gradient-based RL algorithms like REINFORCE.

This is fundamental in RL — agents explore actions stochastically while learning to favor high-reward actions.

b. Token Sampling in Language Models

If you've ever used a text generator like GPT, you've benefited from token sampling. Language models generate text token by token, sampling from a probability distribution over possible next tokens.

Token Sampling Example:

```
import torch
from torch.distributions import Categorical

# Logits output from a language model (unnormalized)
logits = torch.tensor([2.0, 1.0, 0.1])

# Create the categorical distribution
dist = Categorical(logits=logits)

# Sample a token index
token = dist.sample()

# Print the sampled token index
print(f"Sampled token index: {token.item()}")
```

Why Logits?

Most language models output logits instead of probabilities. Using logits, you avoid precision issues and unnecessary computations — PyTorch handles the conversion to probabilities internally.

. . .

Tips for Efficient Sampling

Let's talk about scaling up. If you're working with large-scale data or need to sample multiple values quickly, **torch.multinomial** is your best friend. It's optimized for sampling multiple indices from a probability distribution.

Here's how it works:

```
import torch

# Define probabilities for a single distribution
probs = torch.tensor([0.2, 0.5, 0.3])

# Sample 5 values with replacement
samples = torch.multinomial(probs, num_samples=5, replacement=True)

# Print sampled indices
print(f"Samples: {samples}")
```

Why Use torch.multinomial?

1. **Performance**: Faster for repeated sampling.
2. **Scalability**: Ideal for handling high-dimensional data or batching.

You might notice replacement=True. This means a single index can be selected multiple times in the same sampling run. If your use case requires unique selections, set replacement=False.

. . .

Debugging and Common Pitfalls

Even experts hit roadblocks, and categorical sampling is no exception. Let's address a few common pitfalls and how to avoid them.

Pitfall 1: Negative Probabilities

Probabilities must be non-negative and sum to 1. PyTorch will throw an error if these conditions aren't met.

Example:

```
import torch
from torch.distributions import Categorical

try:
    # Invalid probabilities
    probs = torch.tensor([-0.1, 1.1])
    dist = Categorical(probs)
except Exception as e:
    print(f"Error: {e}")

Output:
javascript
Copy code
Error: ValueError: probs has negative values.
```

How to Fix It: Normalize your probabilities and ensure all values are non-negative:

```
probs = torch.abs(probs) # Make non-negative
probs /= probs.sum() # Normalize to sum to 1
```

Pitfall 2: Floating-Point Precision

Small probabilities or logits can lead to numerical instability. For example, probabilities close to zero might round to zero.

Solution: Use Logits Instead of Probabilities Logits provide better numerical stability, especially when working with large datasets or deep networks. PyTorch handles softmax computation internally, avoiding precision loss.

Pitfall 3: Debugging Batch Dimensions

When sampling from batched distributions, mismatched dimensions can cause errors. Always ensure that your batch dimensions align.

Example:

```
import torch
from torch.distributions import Categorical

# Batch of probabilities
batch_probs = torch.tensor([
    [0.1, 0.7, 0.2],  # Distribution 1
    [0.3, 0.4, 0.3]   # Distribution 2
])

dist = Categorical(batch_probs)

# Correct sampling
samples = dist.sample()
print(f"Samples: {samples}")
```

Summary of Key Insights

- Use Categorical for sampling in RL and NLP tasks where probabilistic decisions are essential.
- Optimize large-scale sampling with torch.multinomial.
- Debug like a pro by validating probabilities, switching to logits when needed, and ensuring batch dimensions align.

Armed with these techniques, you're ready to tackle real-world challenges with confidence. Let's move to the next section, where we'll explore **advanced applications** that push the boundaries of what we've covered so far!

. . .

Advanced Applications

> “Sometimes, the road less traveled is the one worth taking.”

In the world of categorical distributions, advanced use cases often involve creative ways to manipulate probabilities or blend distributions to tackle complex problems. Let’s dive into two fascinating techniques that push the boundaries of what we’ve covered so far.

Custom Sampling Strategies: Reweighting Probabilities Dynamically

You might be wondering, *“What if I want to favor certain outcomes more than others dynamically?”* This is where **reweighting probabilities** comes into play.

By manipulating the probability distribution, you can adjust the sampling behavior to suit your specific needs — be it boosting exploration in RL or emphasizing rare events in generative models.

Example: Scaling Probabilities Quadratically

Here’s how you can modify probabilities dynamically and sample from the adjusted distribution:

```
import torch
from torch.distributions import Categorical

# Original probabilities
probs = torch.tensor([0.1, 0.2, 0.7])

# Dynamically scale probabilities (quadratic scaling in this example)
scaled_probs = probs ** 2

# Normalize the scaled probabilities
normalized_probs = scaled_probs / scaled_probs.sum()

# Create the categorical distribution with reweighted probabilities
dist = Categorical(probs=normalized_probs)

# Sample from the reweighted distribution
sample = dist.sample()

# Print results
print(f"Original Probabilities: {probs}")
```

```
print(f"Scaled Probabilities: {normalized_probs}")
print(f"Reweighted Sample: {sample.item()}")
```

What's Going On?

1. **Scaling**: Each probability is squared, emphasizing larger probabilities more heavily.
2. **Normalization**: The new probabilities are scaled to sum to 1, ensuring they remain valid.
3. **Sampling**: The adjusted distribution is used for sampling, favoring higher-probability outcomes even more strongly.

Use Case Ideas:

- In RL, dynamically adjust probabilities based on state importance.
- In NLP, favor rare words during text generation to add variability.

Continuous Sampling Extension: Combining Categorical with Other Distributions

Life isn't always discrete, and sometimes, you need to blend discrete and continuous elements in your models. For example, think about generating hierarchical data where a discrete choice (e.g., category) determines the parameters of a continuous distribution (e.g., Gaussian).

Let's see how you can combine a Categorical distribution with a Normal distribution:

```
import torch
from torch.distributions import Categorical, Normal

# Define a categorical distribution for discrete choices
probs = torch.tensor([0.3, 0.4, 0.3])
dist_categorical = Categorical(probs)

# Define parameters for Normal distributions for each category
means = torch.tensor([0.0, 5.0, 10.0])
std_devs = torch.tensor([1.0, 2.0, 1.5])

# Sample a category
category = dist_categorical.sample()
```

```
# Use the sampled category to select parameters for a Normal distribution
mean = means[category]
std_dev = std_devs[category]
dist_normal = Normal(mean, std_dev)

# Sample from the selected Normal distribution
sample = dist_normal.sample()

# Print results
print(f"Sampled Category: {category.item()}")
print(f"Mean for Selected Normal: {mean.item()}, Std Dev: {std_dev.item()}")
print(f"Sample from Normal Distribution: {sample.item()}")
```

Why Does This Matter? This approach allows you to model hierarchical dependencies between discrete and continuous variables. For example:

- In reinforcement learning, the category might represent a high-level strategy, and the continuous variable could dictate the fine-tuning of actions.
- In generative modeling, this could represent switching between different modes of a mixture model.

. . .

Conclusion

Congratulations — you’ve just completed a deep dive into sampling from categorical distributions using PyTorch! Let’s recap what we’ve covered:

1. **Basics of Sampling**: From probabilities and logits to batch sampling and gradient tracking.
2. **Practical Use Cases**: Applying Categorical in RL and NLP for real-world problems.
3. **Advanced Applications**: Reweighting probabilities dynamically and blending discrete and continuous distributions for more complex tasks.

Now it’s your turn. Try out the examples we’ve discussed, tweak them for your specific projects, and experiment boldly. The more you explore, the more you’ll discover just how versatile PyTorch’s Categorical distribution can be.

Remember, great machine learning isn't just about following instructions — it's about taking tools like these and creating something uniquely yours. So go ahead, write that RL policy, build that generative model, or solve that next big problem.

And as always, don't forget to share your insights and findings — because the best learning happens when we collaborate.

Follow

Written by whyamit404

114 followers · 17 following

Founder at: https://oneiszero.com/

No responses yet

Write a response

What are your thoughts?

More from whyamit404